

Solicitud de informe de avance y propuesta técnica

SIREGG

A partir del documento de requerimientos entregado, necesito que elaboren un **informe técnico del estado actual del proyecto**, tomando como referencia todo lo que ya han desarrollado hasta la fecha.

El objetivo de este informe es conocer con precisión qué se ha construido, cómo funciona actualmente, qué falta desarrollar y qué mejoras consideran necesarias.

El informe debe incluir, de manera resumida pero suficientemente detallada:

1. Avance actual

- ¿Qué funcionalidades están completamente desarrolladas?
- ¿Qué funcionalidades están parcialmente desarrolladas?
- ¿Qué funcionalidades todavía no se han desarrollado?
- ¿Qué flujo completo puede ejecutarse actualmente de inicio a fin?

2. Arquitectura actual

Explicar cómo está construido actualmente el sistema:

- Frontend.
- Backend.
- Base de datos.
- Almacenamiento de archivos.
- APIs utilizadas.
- Servicios externos.
- Procesamiento de imágenes.
- Procesamiento de audio.
- Automatizaciones.
- Integraciones existentes.

Indicar también cómo se comunican entre sí los diferentes componentes.

3. Tecnologías utilizadas

Indicar las tecnologías, frameworks, librerías, APIs y servicios utilizados actualmente y explicar brevemente para qué se utiliza cada uno.

4. Funcionalidades de inteligencia artificial

Explicar:

- Qué procesos utilizan IA actualmente.
- Qué información puede interpretar el sistema.
- Qué datos extrae automáticamente.
- Qué procesos todavía requieren intervención del usuario.
- Qué limitaciones o problemas han encontrado.

5. Base de datos y estructura de información

Explicar cómo están almacenándose actualmente:

- Gastos.
- Usuarios.
- Empresas.
- Proveedores.
- Comprobantes.
- Pagos.
- Imágenes.
- Audios.
- Pedidos/proyectos.
- Categorías.

Indicar qué información ya está contemplada y qué información consideran que debería incorporarse.

6. Funcionalidades pendientes

Comparar lo desarrollado actualmente con el documento de requerimientos e indicar claramente:

Implementado / Parcial / Pendiente / Requiere modificación

para cada funcionalidad importante.

7. Problemas encontrados

Indicar los principales problemas técnicos o funcionales que hayan encontrado durante el desarrollo y cómo los han solucionado o cómo proponen solucionarlos.

8. Sugerencias del equipo

Quiero que incluyan sus propias propuestas.

Por ejemplo:

- Mejoras de arquitectura.
- Tecnologías alternativas.
- Automatizaciones que consideren convenientes.

- Mejoras en la experiencia del usuario.
- Seguridad.
- Escalabilidad.
- Costos.
- Integraciones futuras.
- Funcionalidades que consideren importantes y que no estén contempladas actualmente.

No se trata solamente de ejecutar los requerimientos; quiero conocer también su criterio técnico.

9. Riesgos y puntos que deberían revisarse

Indicar qué decisiones tomadas hasta ahora podrían generar problemas posteriormente si el sistema crece, especialmente pensando en:

- Multiusuario.
- Multiempresa.
- Mayor volumen de información.
- Integración con ERP.
- Conversión futura a SaaS.
- Seguridad.
- Almacenamiento.
- Costos de APIs y servicios externos.

10. Preguntas y decisiones pendientes

Finalmente, incluir todas las preguntas, dudas o decisiones técnicas que necesiten que definamos antes de continuar.

No asumir decisiones importantes si consideran que requieren validación.

Formato esperado

El informe no necesita ser excesivamente extenso, pero sí debe ser **concreto, técnico y completo**.

La finalidad es tener una fotografía clara del proyecto en este momento:

Qué tenemos → cómo funciona → qué falta → qué problemas existen → qué recomiendan → qué necesitan definir.

Una vez recibido el informe, revisaré sus respuestas y las utilizaré para definir conjuntamente la siguiente etapa del proyecto.

Documento de Requerimientos

Sistema Inteligente de Registro y Gestión de Gastos (SUREGG)

1. Propósito del documento

El presente documento complementa los avances realizados hasta la fecha en el sistema de registro de gastos y formaliza los requerimientos funcionales y técnicos que se han definido durante la evolución del proyecto.

El objetivo no es reemplazar lo desarrollado, sino **ordenar, ampliar y orientar el proyecto hacia una arquitectura que permita inicialmente resolver las necesidades internas de la empresa y posteriormente evolucionar hacia un sistema SaaS comercializable.**

La prioridad inicial es conseguir que el registro de gastos sea extremadamente rápido y sencillo para el usuario:

“Realicé un gasto → envió una foto, audio o mensaje → el sistema interpreta, clasifica, registra y confirma.”

El usuario no debería tener que abrir Excel, buscar una categoría ni realizar posteriormente una clasificación manual masiva.

2. Problema que se busca resolver

Actualmente se realizan diariamente múltiples gastos:

- Compras de materia prima.
- Tinta.
- Papel.
- Tela.
- Herramientas.
- Transporte.
- Servicios.
- Compras menores.
- Compras personales.
- Pagos mediante Yape.
- Pagos mediante transferencia bancaria.
- Compras con boleta.
- Compras con factura.
- Compras sin comprobante.
- Importaciones y compras corporativas.

El problema principal es que los comprobantes, capturas de Yape y demás evidencias se acumulan y posteriormente resulta difícil recordar:

- Qué se compró.
- Cuánto se pagó.
- Para quién o para qué fue la compra.
- Si correspondía a la empresa o a un gasto personal.
- A qué pedido o proyecto correspondía.
- Quién realizó el gasto.
- Si el gasto ya fue registrado por otra persona.

Por ello, el sistema debe priorizar la **captura inmediata y con mínimo esfuerzo**.

3. Concepto general de solución

El sistema deberá permitir registrar un gasto mediante diferentes canales:

1. Fotografía de una boleta o factura.
2. Captura de pantalla de Yape u otro comprobante de pago.
3. Audio enviado por el usuario.
4. Texto escrito.
5. Posteriormente, transferencias, documentos de importación y otros tipos de evidencia.

El sistema deberá interpretar la información mediante IA, normalizarla, clasificarla y almacenarla.

El flujo conceptual será:

Usuario → Canal de captura → IA/OCR → extracción de información → clasificación → detección de duplicados → validación → registro → almacenamiento de evidencias → integración con ERP/contabilidad.

4. Canal de captura

4.1 Primera versión

Para el MVP se recomienda utilizar **Telegram mediante un bot**, debido a que permite una integración sencilla y directa con automatizaciones.

El usuario podrá enviar:

- Foto.
- Audio.
- Texto.
- Captura de pantalla.

Telegram debe considerarse principalmente como **canal de entrada**, no como base documental definitiva.

4.2 Evolución futura

La arquitectura debe permitir agregar posteriormente:

- WhatsApp.
- Aplicación web.
- Aplicación móvil.
- Correo electrónico.
- Otros canales.

La lógica de negocio no debe depender directamente de Telegram.

Por ello, Telegram debe comunicarse con un backend/API propio y este backend debe encargarse de la lógica.

5. Procesamiento de imágenes

Debe contemplarse una arquitectura que permita utilizar modelos de visión mediante API.

El modelo de visión deberá poder interpretar:

- Boletas.
- Facturas.
- Capturas de Yape.
- Comprobantes de pago.
- Fotografías de documentos.
- Documentos parcialmente deteriorados.
- Fotografías con diferentes formatos.
- Documentos manuscritos cuando corresponda.

5.1 OCR

El OCR podrá utilizarse como componente complementario.

No debe asumirse que OCR y visión son lo mismo.

OCR:

- Extrae principalmente caracteres y texto.

Modelo de visión:

- Puede interpretar la estructura y el significado de la imagen.
- Puede identificar qué campo corresponde al total.
- Puede diferenciar fecha, proveedor, subtotal, impuestos, etc.
- Puede interpretar documentos menos estructurados.

La arquitectura debería permitir cambiar o combinar estos componentes sin modificar el resto del sistema.

6. Procesamiento de voz

El usuario podrá enviar mensajes de voz como:

“Compré cinco kilos de mandarina por dieciocho soles para la casa.”

El sistema deberá:

1. Transcribir el audio.
2. Interpretar el contenido.
3. Extraer los datos relevantes.
4. Clasificarlo.
5. Solicitar confirmación únicamente si existe ambigüedad.

Ejemplo:

Personal → Alimentación → S/18 → Mandarina

Otro ejemplo:

“Compré dos metros de tela por ochenta soles para el pedido Dragon.”

Resultado:

Empresa → Materia prima → Tela → Pedido Dragon → S/80

7. Modelo de datos fundamental

Debe evitarse considerar cada documento o comprobante como un gasto independiente.

La estructura conceptual debe separar:

Gasto

Representa la operación económica.

Comprobante

Factura, boleta, recibo u otro documento relacionado.

Pago

Yape, transferencia, efectivo, tarjeta, etc.

Evidencia

Imagen, captura, audio, PDF u otro archivo.

Proyecto/Pedido

Pedido de cliente, producción, importación u otro centro de costo.

Esto permite que una misma operación tenga múltiples evidencias.

Ejemplo:

Gasto: S/50

Factura: factura del proveedor

Pago: Yape

Evidencia: fotografía de factura + captura de Yape

Todo corresponde a **un solo gasto**, no a dos.

8. Clasificación automática

Cada gasto deberá poder clasificarse como mínimo mediante:

Tipo de gasto

- Personal.
- Empresa.

Categoría

Ejemplos:

- Materia prima.
- Tela.
- Tinta.
- Papel.
- Herramientas.
- Transporte.
- Servicios.
- Alimentación.
- Administración.
- Otros.

Centro de costo / proyecto

Ejemplos:

- Pedido Dragon.
- Pedido Colegio X.
- Producción general.
- Administración.
- Importación.
- Personal.

Medio de pago

- Yape.
- Transferencia.
- Efectivo.
- Tarjeta.
- Otro.

9. Clasificación progresiva mediante IA

El sistema debe aprender de las decisiones anteriores.

Ejemplo:

La primera vez:

“Compra a Textiles Pérez.”

El sistema pregunta:

“¿Qué tipo de gasto es?”

Usuario:

“Empresa, tela.”

A partir de ahí podrá establecer una sugerencia:

Textiles Pérez → Empresa → Materia prima → Tela

La próxima vez que aparezca el mismo proveedor, el sistema podrá proponer automáticamente la clasificación.

El usuario deberá poder corregirla.

Las correcciones deberán servir como información para mejorar futuras sugerencias.

10. Minimizar la interacción del usuario

Este es uno de los principios principales del proyecto.

El usuario no debería tener que completar formularios extensos.

Ejemplo ideal:

Usuario envía captura de Yape.

Sistema identifica:

- Monto.
- Fecha.
- Destinatario.
- Número de operación.
- Medio de pago.

Luego pregunta solamente lo que falta:

“¿Qué compraste?”

Usuario:

“Tela para Dragon.”

Sistema:

“Registrado: Empresa / Tela / Pedido Dragon / S/85.”

La IA debe intentar completar automáticamente la mayor cantidad de campos posibles.

11. Gastos personales y empresariales

El sistema debe permitir que un mismo usuario registre gastos personales y empresariales.

Ejemplo:

“Compré pan, tomate y leche por S/27.”

Resultado:

Personal → Alimentación → S/27

Otro:

“Compré cuchillas para producción por S/40.”

Resultado:

Empresa → Herramientas → Producción → S/40

El sistema no debe obligar al usuario a seleccionar manualmente cada categoría cuando la IA pueda determinarla con suficiente confianza.

Cuando exista duda, debe preguntar.

12. Asociación de gastos con pedidos

Los gastos directamente relacionados con una producción deberán poder asociarse a un pedido o proyecto.

Ejemplo:

Pedido Dragon:

- Tela.
- Impresión.
- Corte.
- Confección.
- Transporte.
- Otros.

Posteriormente el sistema deberá poder responder:

“¿Cuánto costó producir el pedido Dragon?”

y consolidar los gastos asociados.

13. Gastos generales e inventario

No todos los gastos deben asociarse directamente a un pedido.

Ejemplo:

Se compra un rollo de papel por S/500 que será utilizado en veinte pedidos.

El gasto inicialmente debe poder registrarse como:

Empresa → Materia prima → Papel → Inventario/Producción general

Posteriormente podrá desarrollarse una asignación proporcional del costo a los diferentes pedidos.

Esta funcionalidad puede considerarse una segunda fase.

14. Factura + Yape

Cuando una compra tenga factura y además exista una captura de Yape, el sistema deberá tratarlos como elementos relacionados de una misma operación.

Ejemplo:

Factura: S/50

Yape: S/50

No deben generarse dos gastos.

La factura representa principalmente el **comprobante de la compra**.

El Yape representa principalmente la **evidencia del pago**.

Por tanto:

1 gasto → 1 factura + 1 pago + 2 evidencias

Cuando únicamente se envíe la factura, deberá ser posible registrar el gasto sin exigir obligatoriamente la captura de Yape.

15. Transferencias bancarias y compras corporativas

El sistema debe contemplar posteriormente operaciones que no utilizan Yape.

Ejemplos:

- Transferencias bancarias.
- Pagos desde banca empresarial.
- Compras corporativas.
- Importaciones.
- Pagos internacionales.
- Gastos bancarios.
- Fletes.
- Seguros.
- Aduanas.
- Impuestos.

Una importación podría agrupar:

- Factura comercial.
- Transferencia internacional.
- Gastos bancarios.
- Flete.
- Seguro.
- Aduanas.
- Impuestos.
- Otros documentos.

Todo ello debería poder asociarse a una misma **operación de compra/importación**.

16. Multiusuario

El sistema deberá ser multiusuario.

Cada registro debe conservar como mínimo:

- Usuario que creó el registro.
- Empresa/organización.
- Fecha y hora.
- Canal utilizado.
- Evidencias asociadas.

Debe ser posible saber quién registró cada operación.

Se deberán contemplar roles posteriormente:

- Propietario.
 - Administrador.
 - Empleado.
 - Contador.
 - Supervisor.
-

17. Detección de duplicados

Este punto es prioritario.

Debe evitarse que una misma compra sea registrada dos veces por diferentes usuarios.

Ejemplo:

Wilber compra cinco kilos de mandarina por S/18 y registra el Yape.

Posteriormente envía la captura a su esposa.

Ella también envía la misma captura al sistema.

El sistema deberá detectar que probablemente se trata de la misma operación.

Para detectar duplicados se deberán comparar diferentes señales:

- Monto.
- Fecha.
- Hora.
- Proveedor/destinatario.
- Número de operación.
- RUC.
- Número de comprobante.
- Imagen.
- Usuario.
- Similitud de contenido.

Si la coincidencia es prácticamente segura:

“Este pago ya fue registrado por Wilber.”

Si existe una coincidencia parcial:

“Posible gasto duplicado. ¿Deseas registrarlo igualmente?”

No se debe eliminar automáticamente una operación cuando solamente existe una similitud parcial.

18. Almacenamiento de evidencias

Las imágenes y audios deben almacenarse independientemente de la base de datos estructurada.

La base de datos debe guardar referencias hacia los archivos.

Ejemplo:

Gasto → ID 4582

Factura → archivo X

Yape → archivo Y

Audio → archivo Z

Para el MVP puede utilizarse almacenamiento tipo Supabase Storage u otra solución equivalente.

Debe contemplarse una política de retención configurable.

Por ejemplo:

- Datos estructurados: conservación permanente.
- Fotografías: conservación de 60 días en un plan básico.
- Audios originales: conservación de 30 días.

Posteriormente, en una versión comercial, la conservación de documentos podrá formar parte del servicio contratado.

No se debe eliminar información financiera simplemente porque el almacenamiento alcanzó su límite. Debe existir una política explícita de retención.

19. n8n

Para el MVP se recomienda utilizar **n8n Community Edition self-hosted** como motor de automatización.

No se recomienda desarrollar inicialmente un “mini-n8n” propio.

n8n puede encargarse inicialmente de conectar:

Telegram → IA → Base de datos → almacenamiento → respuestas.

El sistema propio deberá concentrar la lógica que representa el valor del producto:

- Clasificación.
- Reglas empresariales.
- Detección de duplicados.
- Asociación a pedidos.
- Normalización.
- Usuarios.
- Empresas.
- Centros de costo.
- Reportes.
- Integración con ERP.

Posteriormente, si el producto escala considerablemente, podrá evaluarse reemplazar determinados workflows de n8n por servicios propios.

No debe construirse un motor de automatización propio solamente con el objetivo de ahorrar el costo de n8n.

20. Arquitectura inicial propuesta

La arquitectura conceptual recomendada es:

Canal de entrada

Telegram inicialmente.

↓

n8n

Orquestación de workflows.

↓

Servicios de IA

- Transcripción de voz.
- Modelo de visión.
- Interpretación y estructuración.

↓

Backend propio

Lógica de negocio.

↓

Base de datos

PostgreSQL/Supabase u otra equivalente.



Storage

Fotografías, audios, documentos.



Panel web

Consulta, revisión, corrección y reportes.



ERP / sistema contable

Integración posterior.

La arquitectura debe permitir reemplazar Telegram por WhatsApp sin modificar la lógica central.

21. Integración con ERP

El sistema de gastos debe considerarse un **microservicio o servicio externo especializado**, conectado posteriormente con el ERP de la empresa.

El sistema de captura deberá encargarse de:

- Recibir.
- Interpretar.
- Normalizar.
- Clasificar.
- Validar.
- Consolidar.

El ERP deberá recibir información estructurada y validada.

Esto permite que el sistema de gastos pueda conectarse posteriormente con diferentes ERPs sin quedar acoplado a uno específico.

22. Normalización de información

La IA puede interpretar diferentes formas de decir lo mismo.

Ejemplo:

“tinta”

“tinta sublimación”

“tinta para impresora”

“ink sublimación”

El sistema debe poder normalizar estos conceptos bajo una estructura común.

Lo mismo con proveedores, categorías, medios de pago y proyectos.

La información almacenada debe ser estructurada y consistente, aunque el usuario se exprese de manera informal.

23. Confianza y revisión humana

La IA no debe asumir que todo resultado es correcto.

Cada extracción puede tener un nivel de confianza.

Conceptualmente:

Alta confianza: registrar automáticamente.

Confianza media: solicitar confirmación.

Baja confianza: solicitar información al usuario.

Esto permitirá reducir progresivamente el trabajo humano sin perder control.

24. Panel de control

El sistema deberá evolucionar hacia un dashboard que permita consultar:

- Gastos del día.
- Gastos de la semana.
- Gastos del mes.
- Gastos personales.

- Gastos empresariales.
- Gastos por categoría.
- Gastos por proveedor.
- Gastos por usuario.
- Gastos por proyecto.
- Gastos por medio de pago.
- Gastos pendientes de revisión.
- Posibles duplicados.
- Gastos sin comprobante.

Posteriormente:

- Costos por pedido.
 - Rentabilidad por pedido.
 - Consumo de materiales.
 - Presupuesto vs. gasto.
 - Conciliación bancaria.
 - Información contable.
-

25. Evolución hacia SaaS

Aunque la primera versión sea para uso interno, la arquitectura debe evitar decisiones que impidan convertirla posteriormente en SaaS.

El sistema debería poder evolucionar hacia:

Empresa A

- Usuarios.
- Gastos.
- Proyectos.
- Proveedores.
- Evidencias.

Empresa B

- Usuarios.
- Gastos.
- Proyectos.
- Proveedores.
- Evidencias.

Los datos deben mantenerse completamente aislados entre organizaciones.

Se deberá utilizar un modelo multi-tenant desde una etapa temprana si el costo técnico es razonable.

26. Posible propuesta de valor futura

El producto no debería venderse como:

“Sistema OCR.”

Ni como:

“Sistema de n8n.”

La propuesta debería estar orientada al problema:

“Registra tus gastos simplemente enviando una foto, audio o mensaje.”

El sistema debe encargarse automáticamente de:

- Leer el comprobante.
- Identificar el monto.
- Identificar proveedor.
- Identificar fecha.
- Identificar medio de pago.
- Clasificar el gasto.
- Asociarlo a empresa o personal.
- Asociarlo a proyecto.
- Detectar posibles duplicados.
- Guardar la evidencia.
- Preparar la información para contabilidad.

27. Fases recomendadas

Fase 1 — MVP interno

Objetivo: demostrar el flujo básico.

- Telegram.
- Foto.
- Audio.
- Texto.
- IA.
- PostgreSQL/Supabase.
- Storage.

- Clasificación básica.
- Confirmación.
- Registro.

Fase 2 — Inteligencia

- Aprendizaje de proveedores.
- Clasificación automática.
- Detección de duplicados.
- Asociación a pedidos.
- Reglas personal/empresa.
- Mayor automatización.

Fase 3 — Gestión

- Dashboard.
- Usuarios.
- Roles.
- Reportes.
- Exportación.
- Gestión de comprobantes.
- Centros de costo.

Fase 4 — Integraciones

- WhatsApp.
- ERP.
- Sistemas contables.
- Bancos.
- SUNAT u otras fuentes disponibles.
- Importaciones.

Fase 5 — SaaS

- Multiempresa.
- Suscripciones.
- Límites por plan.
- Almacenamiento por plan.
- Usuarios por plan.
- API.
- Seguridad.
- Auditoría.
- Facturación.
- Panel administrativo.

28. Principios que deben mantenerse durante el desarrollo

1. El registro debe tomar segundos, no minutos.
 2. La IA debe hacer el trabajo pesado de interpretación.
 3. El usuario debe intervenir solamente cuando exista ambigüedad.
 4. Una operación económica no debe duplicarse por tener varios documentos.
 5. Las evidencias deben estar vinculadas al gasto.
 6. Personal y empresa deben mantenerse separados.
 7. Los gastos deben poder asociarse a pedidos/proyectos.
 8. Los datos estructurados deben conservarse independientemente de las imágenes.
 9. Telegram/n8n/IA deben ser componentes reemplazables.
 10. La arquitectura debe permitir evolucionar hacia un SaaS multiempresa.
 11. No desarrollar infraestructura propia cuando exista una solución madura que resuelva esa función.
 12. El esfuerzo de desarrollo debe concentrarse en la lógica de negocio que diferencia al producto.
-

29. Prioridad para el equipo actual

Dado que ya existe desarrollo avanzado, **no se recomienda reiniciar el proyecto.**

Se deberá revisar lo actualmente desarrollado y determinar:

- Qué requerimientos ya están implementados.
- Qué requerimientos están parcialmente implementados.
- Qué componentes deben modificarse.
- Qué componentes pueden mantenerse.
- Qué componentes deben agregarse.
- Qué decisiones actuales podrían dificultar una futura evolución a SaaS.

El objetivo es **complementar y mejorar el desarrollo existente**, no desecharlo.

La primera meta concreta debe ser conseguir un flujo completo y funcional:

Foto/audio → interpretación → clasificación → detección de duplicidad → confirmación → registro → almacenamiento de evidencia → consulta.

Una vez validado este flujo con gastos reales, se deberá avanzar hacia multiusuario, pedidos, ERP y posteriormente SaaS.

